The Spacetime Class Library

Joe Boudreau 2017

Contents

1 Introduction

The `Spacetime` Class Library is a small collection of classes intended to facilitate numerical computations in nonrelativistic and relativistic quantum mechanics. The library contains rotations and Lorentz transformation classes, instances of which can provide matrix representations of the corresponding group element. Objects such as vectors, spinors, and Dirac spinors (among others) which belong to some representation of the rotation group (or the Lorentz group) are transformed accordingly. The design allows other types of covariant objects to be introduced into the library at a later time. Presently, the library contains vectors ($\vec{v}$), four-vectors (p^μ), spinors ($|\uparrow\rangle$, $|\downarrow\rangle$), Dirac spinors (u,v,$\bar{u}$, $\bar{v}$) and Weyl spinors, and special operators such as Pauli matrices ($\vec{\sigma} = \sigma_x, \sigma_y \sigma_z$), gamma matrices, $\gamma^0, \gamma^1, \gamma^2, \gamma^3$, and the matrices $S^{\mu\nu} = \frac{i}{4}[\gamma^\mu, \gamma^\nu]$. The usual operations are defined on these classes. The library depends upon the `Eigen` library for matrix manipulation[1]. This documentation is the reference manual for the `Spacetime` library.

2 Rotations and Lorentz Tranformations

2.1 class Rotation

```
#include "Spacetime/Rotation.h"
```

Description
A rotation is specfied by an axis and an angle, interpreted as an active right-handed rotation. For example, a rotation about the $+z$ axis carries the x axis into the y-axis and the y axis into the $-x$-axis. A rule of composition applies to rotations and is expressed with the multiplication symbol (operator *). The rotation also provides a matrix in any desired $D = 2j+1$ dimensional irreducible representation of SU(2), expressed in the basis$\{\vec{e}_0 = |j, j\rangle$, $\vec{e}_1 = |j, j-1\rangle$.. $\vec{e}_{2j} = |j, -j\rangle\}$. Here, j is the angular momentum quantum number, integer or half-integer. Rotations can also act on other objects like vectors and spinors, but this action is defined in free subroutines, which are discussed in conjunction with vectors, spinors, and other mathematical objects upon which the action of the rotation is defined. These are described later.

Methods
Constructor. Constructs an identity rotation:

- `Rotation()`

Constructor. Constructs a rotation from a vector. The length of the vector represents the angle of rotation, in radians. the direction represents the axis of rotation.

[1] http://eigen.tuxfamily.org

- `Rotation(const ThreeVector & axisAngle)`

Composition

- `operator * (const Rotation & right)`

Retrieve the rotation matrix in the D-dimensional representation of SU(2). Here $D = 2j+1$ where j is the angular momentum quantum number, integer or half-integer.

- `Eigen::MatrixXcd & rep(unsigned int d) const`

Get the rotation vector, whose length is equal to the angle of rotation in radians and whose direction represents the axis of rotation (right-handed, active).

- `const ThreeVector & getRotationVector() const`

Return the inverse rotation:

- `Rotation inverse() const`

2.2 class LorentzTransformation

`#include "Spacetime/LorentzTransformation.h"`

Description
The LorentzTransformation class represents a proper, orthochronous Lorentz transformation. It is specified by two vectors, the first representing the boost and the second representing the rotation vector. Both are to be considered as active, i.e. acting on the body rather than the coordinate system. The boost vector points along the direction of the boost and has a magnitude of $\eta = \tanh^{-1}\beta$ (Achtung!) where $\beta = v/c$, v is the velocity and c is the speed of light. The rotation vector is described in section 2.1. Irreducible representations of the proper orthochronous Lorentz group are equivalent to $SU(2)\times SU(2)$, and accordingly two matrices are given, one for each of the two $SU(2)$ subgroups. Composition of LorentzTransformations can be carried out with the * operator.

Methods
Constructor. Constructs an identity element.

- `LorentzTransformation();`

Constructor. Takes the rapidity vector and the rotation vector

- ```
  LorentzTransformation(
  const ThreeVector & rapVector,
  const ThreeVector & rotVector=ThreeVector(0,0,0))
  ```

Composition.

- ```
  LorentzTransformation operator * (
  const LorentzTransformation & source);
  ```

Get the D-dimensional representation, matrix 1 and matrix 2

- `const Eigen::MatrixXcd & rep1(unsigned int dim) const;`
- `const Eigen::MatrixXcd & rep2(unsigned int dim) const;`

Get the rotation vector and the rapidity vector

- `ThreeVector getRotationVector() const`
- `ThreeVector getRapidityVector() const`

Get the inverse.

- `LorentzTransformation inverse() const`

3 Vectors, Four-Vectors, and Tensors

This section describes vectors in Cartesian 3-space, 4-vectors in Minkokski space, and the assymetric four-tensor, which is an tensor having simple properties under Lorentz tranformations. These objects may be transformed by rotations and/or Lorentz transformations. The work is done in free subroutines, which are also described in this section.

3.1 template <class T> BasicThreeVector

`#include "Spacetime/ThreeVector.h"`

Description
A `BasicThreeVector` is the classic vector in Cartesian 3-space. It is affected by rotations. It is implemented as a template class because in addition to real vectors, we sometimes need complex vectors, to describe certain polarization states, for example. Two parameterized classes are also defined in the header, `ThreeVector` and `ComplexThreeVector`.

Methods
Constructor. Constructs a zero vector

- `BasicThreeVector()`

Constructor. Constructs from components

- `BasicThreeVector(T x0, T x1, T x2)`

Access to elements

- `const T & operator[] (unsigned int i) const`
- `T & operator[] (unsigned int i)`

Access to elements:

- `const T & operator() (unsigned int i) const`
- `T & operator() (unsigned int i)`

Cross and dot:

- `BasicThreeVector<T> cross(const BasicThreeVector<T> & right) const`
- `T dot(const BasicThreeVector<T> & source) const`

Compound operations

- `BasicThreeVector<T> & operator +=( const BasicThreeVector<T> & right)`
- `BasicThreeVector<T> & operator -=( const BasicThreeVector<T> & right)`
- `BasicThreeVector<T> & operator *=(T s);`
- `BasicThreeVector<T> & operator /=(T s)`

Unary minus

- `BasicThreeVector<T> operator- () const`

Conjugate

- `BasicThreeVector<T> conjugate() const`

Norm

- `double norm() const`

Squared norm:

- `double squaredNorm() const`

Normalized version of four-vector.

- `BasicThreeVector<T> normalized() const`

Parameterized types

- `typedef BasicThreeVector<double> ThreeVector`
- `typedef BasicThreeVector<std::complex<double>> ComplexThreeVector`

Other operations
Rotation

- `ThreeVector operator *( const Rotation & rot, const ThreeVector & v)`
- `ComplexThreeVector operator *( const Rotation & rot, const ComplexThreeVector & v )`

Formatted I/O

- `template <typename T> std::ostream & operator << (`
  `std::ostream & o,`
  `const BasicThreeVector<T> &v)`

Vector addition and subtraction

- `template <typename T> BasicThreeVector<T> operator+(`
  `const BasicThreeVector<T> & a,`
  `const BasicThreeVector<T> & b)`
- `template <typename T> BasicThreeVector<T> operator-(`
  `const BasicThreeVector<T> & a,`
  `const BasicThreeVector<T> & b)`

Scaling:

- `template <typename T> BasicThreeVector<T> operator *(`
  `const T s,`
  `const BasicThree Vector<T> & v)`
- `template <typename T> BasicThreeVector<T> operator *(`
  `const BasicThreeVector<T> & v,`
  `T s)`
- `template <typename T> BasicThreeVector<T> operator /(`
  `const BasicThreeVector<T> & v,`
  `T s)`

3.2 template class<T> BasicFourVector

`#include "Spacetime/FourVector.h"`

Description
A `BasicFourVector` is a four-component vector in Minkowski space. It is affected by Lorentz transformations. It is implemented as a template class because in addition to real four-vectors, we sometimes need complex four-vectors, to describe circular polarization states. Two parameterized classes are also defined in the header, `FourVector` and `ComplexFourVector`.

Methods
Construct a null four vector

- `BasicFourVector()`

Construct from four components

- `BasicFourVector(T x0, T x1, T x2, T x3)`

Access to elements

- `const T & operator[] (unsigned int i) const`
- `T & operator[] (unsigned int i)`

Access to elements

- `const T & operator() (unsigned int i) const`
- `T & operator() (unsigned int i)`

Return the space portion

- `BasicThreeVector<T> space() const`

Returns the invariant interval

- `T dot(const BasicFourVector<T> & source) const`

Compound operations

- `BasicFourVector<T> & operator +=(const BasicFourVector<T> & right)`
- `BasicFourVector<T> & operator -=(const BasicFourVector<T> & right)`
- `BasicFourVector<T> & operator *=(T s)`
- `BasicFourVector<T> & operator /=(T s)`

Unary minus

- `BasicFourVector<T> operator- () const`

Conjugate

- `BasicFourVector<T> conjugate() const`

Norm

- `double norm() const`

Squared norm:

- `double squaredNorm() const`

Parameterized types

- `typedef BasicFourVector<double> FourVector`
- `typedef BasicFourVector<std::complex<double>> ComplexFourVector`

Other operations
Lorentz Transformations

- ```
  FourVector operator *(
  const LorentzTransformation & transform,
  const FourVector & v)
  ```
- ```
  ComplexFourVector operator *(
  const LorentzTransformation & transform,
  const ComplexFourVector & v)
  ```

Formatted I/O

- ```
  template <typename T> std::ostream & operator << (
  std::ostream & o,
  const BasicFourVector<T> &v)
  ```

Four-vector addition and subtraction

- ```
  template <typename T> BasicFourVector<T> operator+(
  const BasicFourVector<T> & a,
  const BasicFourVector<T> & b)
  ```
- ```
  template <typename T> BasicFourVector<T> operator-(
  const BasicFourVector<T> & a,
  const BasicFourVector<T> & b)
  ```
  ```

Scaling:

- `template <typename T> BasicFourVector<T> operator *(`
  `const T s,`
  `const BasicFourVector<T> & v)`
- `template <typename T> BasicFourVector<T> operator *(`
  `const BasicFourVector<T> & v,`
  `T s)`
- `template <typename T> BasicFourVector<T> operator /(`
  `const BasicFourVector<T> & v,`
  `T s)`

3.3 class AntisymmetricFourTensor

`#include "Spacetime/AntisymmetricFourTensor.h"`

Description

An antisymmetric four-tensor is a tensor in Minkowski space with simple transformation properties under Lorentz transformations. This class has the property that modifying any one of its elements T^{ij} automatically changes T^{ji} to $-T^{ij}$; this works thanks to the helper class `ddouble` which you can be used exactly as a `double`. This works because of a cast operator that permits automatic type conversion. The best known example of an antisymmetric four-tensor is the Maxwell field tensor.

Methods Construct a null tensor:

- `AntisymmetricFourTensor()`

Access to elements. Class ddouble is used just like a double.

- `const ddouble & operator() (unsigned int i, unsigned int j) const`
- `ddouble & operator() (unsigned int i, unsigned int j)`

Compound operations

- `AntisymmetricFourTensor & operator +=(`
  `const AntisymmetricFourTensor & right)`
- `AntisymmetricFourTensor & operator -=(`
  `const AntisymmetricFourTensor & right)`
- `AntisymmetricFourTensor & operator *=(double s)`

Unary minus:

- `AntisymmetricFourTensor operator- () const`

Other operations

Formatted output:

- std::ostream & operator << (
  std::ostream & o,
  const AntisymmetricFourTensor &v)

Arithmetic operations:

- `AntisymmetricFourTensor operator+(`
  `const AntisymmetricFourTensor & a,`
  `const AntisymmetricFourTensor & b)`
- `AntisymmetricFourTensor operator-(`
  `const AntisymmetricFourTensor & a,`
  `const AntisymmetricFourTensor & b)`
- `AntisymmetricFourTensor operator *(`
  `const double s,`
  `const AntisymmetricFourTensor & v)`
- `AntisymmetricFourTensor operator *(`
  `const AntisymmetricFourTensor & v,`
  `double s)`

Lorentz Transformation:

- `AntisymmetricFourTensor operator *(`
  `const LorentzTransformation & transform,`
  `const AntisymmetricFourTensor & v)`

4 Spinors

This section describes three types of spinors (in a D-dimensional Hilbert space), Dirac spinors, and Weyl spinors.

4.1 template <unsigned int D=2> Spinor

`#include "Spacetime/Spinor.h"`

Description

This class represents a spinor in a $D = 2j + 1$ dimensional space. By default $D = 2$ and the spinor describes spin $j = 1/2$ particles.

Methods

Construct a null spinor:

- `Spinor()`

Construct a spinor from an initializer list of `std::complex<double>`:

- `Spinor(const std::initializer_list<std::complex<double>> & values)`

Construct a spinor from other elgible datatypes from the `Eigen` library:

- `template<typename Derived> Spinor(`
  `const Eigen::MatrixBase<Derived> & other)`

Assign Eigen expressions to the spinor:

- `template<typename Derived> Spinor<D> & operator = (`
  `const Eigen::MatrixBase<Derived> & other)`

Other operations

Action of Rotation upon Spinor:

- `template <unsigned int D> Spinor<D> operator * (`
  `const Rotation & R,`
  `const Spinor<D> & v)`

Recover the spin from the Spinor:

- `template <unsigned int D> ThreeVector spin(const Spinor<D> & s)`

4.2 Weyl spinor classes

`#include "Spacetime/WeylSpinor.h"`

Description Weyl spinors are two-component spinors representing massless particles, transforming under rotations and Lorentz transformations. The following datatypes are defined within the namespace `WeylSpinor`:

- `template <int T> BasicSpinor` is a complex two-component column vector. It is essentially `Eigen::Vector2cd` plus some additional constructors. The template parameter determines whether the spinor is of type *right* or *left*.
- `enum Type {RightHandedType, LeftHandedType}` is for selecting right- or left-handed spinors (at compile time).
- `typedef BasicSpinor<RightHandedType> Right`
- `typedef BasicSpinor<LeftHandedType> Left`

Operations Action of Rotation upon the two types of spinor:

- ```
  WeylSpinor::Left operator * (
  const LorentzTransformation & L,
  const WeylSpinor::Left & s)
  ```
- ```
  WeylSpinor::Right operator * (
  const LorentzTransformation & L,
  const WeylSpinor::Right & s)
  ```

Return the Four-momentum of this Weyl Spinor

- `FourVector fourMomentum(const WeylSpinor::Left &s)`
- `FourVector fourMomentum(const WeylSpinor::Right &s)`

Return the Spin (magnitude and direction)

- `ThreeVector spin(const WeylSpinor::Left &s)`
- `ThreeVector spin(const WeylSpinor::Right &s)`

4.2.1 template <int T> class BasicSpinor

The basic 2-component Weyl spinor

Inherits `Eigen::Vector2cd`

Methods Constructs a null spinor:

- `BasicSpinor()`

Constructor from momentum vector:

- `BasicSpinor(const ThreeVector & p)`

Construct a spinor from other elgible datatypes from the `Eigen` library:

- `template<typename Derived> BasicSpinor(`
  `const Eigen::MatrixBase<Derived>& other)`

Assign a spinor from other elgible datatypes from the `Eigen` library:

- `template<typename Derived> BasicSpinor & operator= (`
  `const Eigen::MatrixBase <Derived>& other)`

4.3 Dirac spinor classes

`#include "Spacetime/DiracSpinor.h"`

Description Dirac spinors describe four-component complex column vectors with specific transformation properties under rotations and Lorentz transformations. In this library the internal representation is the Weyl, or chiral, representation. The following datatypes are defined in the `namespace Dirac`:

- `template <int T> BasicSpinor` is a complex four-component column vector. It is essentially `Eigen::Vector4cd` plus some additional constructors. The template parameter determines whether the spinor is of u-type (representing fermions) or of v-type (antifermions).
- `class Any` will hold either a u-type or a v-type Dirac spinor. It too is a complex four-component column vector an essentially an `Eigen::Vector4cd`, with a few extra constructors, but minus special constructors for building the specific u- or v-types.
- `class Bar.` This is a complex four component *row* vector. It is essentially an `Eigen::RowVector4cd`, plus some additional constructors.
- `enum Type {UType, VType}` is for selecting u-type or v-type spinors.
- `typedef BasicSpinor<UType> U`
- `typedef BasicSpinor<VType> V`

Operations Take the spinor adjoint (spinor-bar):

- `DiracSpinor::Bar bar(const DiracSpinor::U & u)`
- `DiracSpinor::Bar bar(const DiracSpinor::V & v)`

Action of Lorentz transformation upon the spinor

- ```
  DiracSpinor::U operator * (
  const LorentzTransformation & L,
  const DiracSpinor::U & u)
  ```
- ```
  DiracSpinor::V operator * (
  const LorentzTransformation & L,
  const DiracSpinor::V & v)
  ```

Return the four-momentum of this Dirac spinor

- `FourVector fourMomentum(const DiracSpinor::U &u)`
- `FourVector fourMomentum(const DiracSpinor::V &v)`

Return the spin (magnitude and direction)

- `ThreeVector spin(const DiracSpinor::U &u)`
- `ThreeVector spin(const DiracSpinor::V &v)`

4.3.1 template<int T> BasicSpinor

The basic 4-component column spinor.

Inherits `Eigen::Vector4cd`

Methods: Construct a null Dirac spinor:

- `BasicSpinor()`

Construct a Dirac spinor of a particle at rest with spin described by the `Spinor s` and mass `m` :

- `BasicSpinor(const Spinor<2> & s, double mass)`

Construct a Dirac spinor of a particle with spin described by the `Spinor s` and with four-momentum `p`:

- `BasicSpinor(const Spinor<2> & s, const FourVector & p)`

Construct a Dirac spinor for a massless particle from the corresponding Weyl spinor:

- `BasicSpinor(const WeylSpinor::Left & s)`
- `BasicSpinor(const WeylSpinor::Right & s)`

Construct a Dirac spinor from other elgible datatypes from the `Eigen` library:

- `template<typename Derived> BasicSpinor(`
  `const Eigen::MatrixBase<Derived>& other)`

Assign a Dirac spinor from other elgible datatypes from the `Eigen` library:

- `template<typename Derived> BasicSpinor & operator= (`
  `const Eigen::MatrixBase <Derived>& other)`

4.3.2 class Bar

The basic 4-component row spinor.

Inherits `Eigen::RowVector4cd`

Methods: Construct a null Dirac spinor-bar:

- `Bar()`

Construct a Dirac spinor-bar from four numbers:

- ```
  Bar(
  const std::complex<double> & s0,
  const std::complex<double> & s1,
  const std::complex<double> & s2,
  const std::complex<double> & s3 )
  ```

Construct a Dirac spinor-bar from other elgible datatypes from the `Eigen` library:

- ```
  template<typename Derived> Bar(
  const Eigen::MatrixBase<Derived>& other)
  ```

Assign a Dirac spinor-bar from other elgible datatypes from the `Eigen` library:

- ```
  template<typename Derived> Bar & operator= (
  const Eigen::MatrixBase <Derived>& other)
  ```

4.3.3 class Any

Represents a generic Dirac Spinor without a specific flavor (u-type or v-type):

Inherits: `Eigen::Vector4cd`

Methods: Construct a generic Dirac spinor from other elgible datatypes from the `Eigen` library:

- ```
  template<typename Derived> Any(
  const Eigen::MatrixBase<Derived>& other)
  ```

Assign a generic Dirac spinor from other elgible datatypes from the `Eigen` library:

- ```
  template<typename Derived> Any & operator= (
  const Eigen::MatrixBase <Derived>& other)
  ```
  ```

5 Operators

Operators are defined in two header files, `Operator4.h` and `SpecialOperators.h`. `Operator4.h` defines some generic interfaces to four-vector operators like $\left(\gamma^0, \gamma^1, \gamma^2, \gamma^3\right)$, and four-tensor operators like $\sigma^{\mu\nu} \equiv \frac{i}{2}[\gamma^u, \gamma^\nu]$. `SpecialOperators.h` defines objects like the Pauli matrices, the γ matrices, the Feynman slash function, etc., and two classes (`Gamma` and `Sigma4x4`) providing implementations of the aforementioned "generic interfaces".

5.1 class AbsOperator4

`#include "Spacetime/Operator4.h"`

Description

An `AbsOperator4` in this library is a four-vector *operator*. Examples of this include

- $\left(\gamma^0, \gamma^1, \gamma^2, \gamma^3\right)$
- $\left(\sigma^0, \sigma^1, \sigma^2, \sigma^3\right)$

`AbsOperator4` is an abstract base class for these operators. It specifies two operations, access to elements, and virtual copy constructor (clone).

Methods

Readonly access to elements:

- `virtual const Eigen::Matrix4cd & operator[](`
  `unsigned int i) const = 0`

Clone:

- `virtual const AbsOperator4 *clone() const=0`

Other operations:

- `Operator4 operator*(`
  `const std::complex<double> &,`
  `const AbsOperator4 &)`
- `Operator4 operator*(`
  `const AbsOperator4 &,`
  `const std::complex<double> &)`

- `Operator4 operator*(`
  `const Eigen::Matrix4cd &,`
  `const AbsOperator4 &)`
- `Operator4 operator*(`
  `const AbsOperator4 &,`
  `const Eigen::Matrix4cd &)`
- `Operator4 operator+(`
  `const AbsOperator4 &,`
  `const AbsOperator4 &)`
- `Operator4 operator-(`
  `const AbsOperator4 &,`
  `const AbsOperator4 &)`

5.2 class Operator4

`#include "Spacetime/Operator4.h"`

Description

`Operator4` is an implementation of `AbsOperator4`, which stores each of the four four-dimensional complex operators in an array of complex matrices. The Weyl representation is used.

Methods

Readonly access to elements:

- `virtual const Eigen::Matrix4cd & operator[](unsigned int i) const`

Read/write access to the elements:

- `Eigen::Matrix4cd & operator[](unsigned int i)`

Clone:

- `virtual const Operator4 *clone() const`

5.3 class AbsOperator4x4

`#include "Spacetime/Operator4.h"`

Description

`AbsOperator4x4` is an abstract base class for tensor operators, such as $\sigma^{\mu\nu} \equiv \frac{i}{2}[\gamma^u, \gamma^\nu]$, which currently is implemented by `class Sigma4x4` which lives in `SpecialOperators.h`, and is the only subclass of `AbsOperator4x4` implemented so far.

Methods

Readonly access to elements:

- ```
  virtual const Eigen::Matrix4cd & operator()(
  unsigned int i,
  unsigned int j) const = 0
  ```

Clone:

- `virtual const AbsOperator4x4 *clone() const=0`

Take a dot product with a four vector to return an Operator 4, eg sigma.dot(q)

- `Operator4 dot(const FourVector & p)`
  ```

5.4 class SU2Generator

`#include "Spacetime/SU2Generator.h"`

Description

`SU2Generator` is a class with no instances, only static member functions. It provides the generators J_x, J_y, and J_z in $D = 2j+1$ dimensions. It also includes facilities to obtain the rotation matrix, or Drehung-matrix, or D-matrix, for a rotation about any specified axis, and to go backwards from a D-matrix to its logarithm.

static member functions

Access to the generators, held in cache:

- `static const Eigen::MatrixXcd & JX(unsigned int dim)`
- `static const Eigen::MatrixXcd & JY(unsigned int dim)`
- `static const Eigen::MatrixXcd & JZ(unsigned int dim)`

Exponentiates linear combinations of SU2 generators in an arbitrary dimensional space to form a rotation (Drehung) matrix, which is a rotation around the axis `nHat` by the angle `phi`.

- ```
  static Eigen::MatrixXcd exponentiate(
  int phi,
  const Eigen::Vector3cd & nHat)
  ```

Takes the logarithm of a rotation (Drehung) matrix

- `static Eigen::MatrixXcd logarithm(const Eigen::MatrixXcd &source)`
  ```

5.5 Special Operators

`#include "Spacetime/SpecialOperators.h"`

Description

The header file `SpecialOperators.h` defines several objects like Pauli spin matrices, gamma matrices, and also a few objects that give a friendlier interface to those; plus functions for the Feynman slash notation. These are scoped within the namespace `SpecialOperators`.

const objects defined with the namespace SpecialOperators

Pauli spin matrices:

- `const Eigen::Matrix2cd sigma0, sigma1, sigma2, sigma3`

Gamma matrices (Weyl representation is used):

- `const Eigen::Matrix4cd gamma0, gamma1, gamma2, gamma3, gamma5`

Array of first four gamma matrices. See also `class Gamma`:

- `const Eigen::Matrix4cd gamma[4]`

Projection operators to left-and right-handed chirality states:

- `const Eigen::Matrix4cd PL, PR`

4x4 array of sigma matrices and spin matricies ($S = \sigma/2$). See also class `Sigma4x4`.

- `const Eigen::Matrix4cd sigma[4][4]`
- `const Eigen::Matrix4cd S[4][4]`

Other operations

Feynman slash notation:

- `inline Eigen::MatrixXcd slash(const FourVector & p)`
- `inline Eigen::MatrixXcd slash(const ComplexFourVector & p)`

5.5.1 class Gamma

`#include "SpecialOperators.h"`

description Access to gamma matrices γ^0, γ^1, γ^2 and γ^3. The `clone` method allows the construction of four-currents.

inherits `AbsOperator4`

methods Clone:

- `virtual const Gamma *clone() const`

Return one of the gamma matrices

- `inline const Eigen::Matrix4cd & operator[] (unsigned int i) const`

5.5.2 class Sigma4x4

#include "SpecialOperators.h"

description Provides access to the matrices $\sigma^{\mu\nu} \equiv \frac{1}{2}[\gamma^\mu, \gamma^\nu]$. The clone method allows the construction of four-currents.

inherits `Operator4x4`

methods Clone:

- `virtual const Sigma4x4 *clone() const`

Return one of the gamma matrices:

- ```
  const Eigen::Matrix4cd & operator() (
  unsigned int i,
  unsigned int j) const
  ```

Contract with a four-vector to obtain a four-vector operator:

- `Operator4 dot(const FourVector & p) const`
  ```

5.6 class Current4

`#include "Spacetime/Current4.h"`

description The `Current4` class facilitates the construction of expressions like $\bar{u}(k')\gamma^\mu u(k)$, $\bar{u}(k')\sigma^\mu_\nu\gamma^\nu u(k)$, etc. In many cases it is necessary to explicitly create any instances of `Current4` because temporary `Current4` objects will be created automatically when expresssions such as the above examples are written naturally. The header file must nonetheless be included. This class has no methods, and two friends:

operations:

- ```
  Current4 operator * (
  const DiracSpinor::Bar &,
  const AbsOperator4 &)
  ```
- ```
  Current4 operator * (
  const Current4 &,
  const DiracSpinor::Any & )
  ```

6 A typical expression

With few classes, you can do a lot of fundamental physics. In this section, we give a brief example. In Compton scattering, one has to evaluate the following matrix elements:

$$-i\mathcal{M}_s = \bar{u}' \left[\epsilon'^*_\nu (ie\gamma^\nu) \frac{i(\not{p} + \not{k} + m_e)}{(p+k)^2 - m_e^2} (ie\gamma^\mu) \epsilon_\mu \right] u$$

(s-channel matrix element), and

$$-i\mathcal{M}_t = \bar{u}' \left[\epsilon_\mu (ie\gamma^\mu) \frac{i\ (\not{p} - \not{k}' + m_e)}{(p-k')^2 - m_e^2} (ie\gamma^\nu) \epsilon'^*_\nu \right] u,$$

(t-channel matrix element); these can be expressed in the Spacetime library as

```
DiracSpinor uIn, uOut;
FourVector p, k, kPrime;
ComplexFourVector epsIn, epsOut;
.
.
.
std::complex<double> MS =
  e*e*
  ((bar(uOut)*
  (slash(epsOut.conjugate()))*
  (slash(p+k)+M)/((p+k).squaredNorm()-m*m)*
  slash(epsIn))*
  uIn)
  (0));

std::complex<double>  MT =
  e*e*
  ((bar(uOut)*
  (slash(epsIn)*
  (slash(p-kPrime)+M)/((p-kPrime).squaredNorm()-m*m)*
  slash(epsOut.conjugate()))*
  uIn)
(0));
```

after proper initialization of all `DiracSpinors`, `FourVectors`, and `ComplexFourVectors` entering into the expression. Notice that `Current4` class temporaries are created so the code will not compile unless the `Current4` header file is included.